



HƯỚNG DẪN PHÁT TRIỂN WEB CHO NGƯỜI MỚI BẮT ĐẦU - UCVN

Thư viện số



CÔNG TY TNHH GIẢI PHÁP PHẦN MỀM UC VIỆT NAM

HÀ NỘI, THÁNG 11 / 2024

MỤC LỤC

1	Tổng quan tài liệu	3
1.1	Mục đích của tài liệu	3
1.2	Đối tượng đọc tài liệu	3
1.3	Thông tin phát triển.....	3
1.3.1	Ngôn ngữ lập trình, thư viện, framework.....	3
1.3.2	Cơ sở dữ liệu.....	4
1.3.3	UCVN helper	4
1.3.4	Công cụ và môi trường phát triển.....	4
2	Hướng dẫn lập trình API	5
2.1	Cấu trúc dự án	5
2.2	Quy tắc chung.....	5
2.2.1	Quy tắc đặt tên :	5
2.2.2	Quy tắc tạo Entity :	5
2.2.3	Quy tắc viết API :	5
2.3	UCHelper	5
2.4	Dapper cơ bản	6
2.4.1	Truy vấn dữ liệu :	6
2.4.2	Thao tác xử lý dữ liệu :	6
2.5	UC Query Builder.....	7
2.6	CRUD API cơ bản cho 1 Entity.....	7
3	Hướng dẫn lập trình giao diện.....	10
3.1	Cấu trúc dự án	10
3.2	Quy tắc chung.....	10
3.2.1	Quy tắc đặt tên :	10
3.2.2	Quy tắc code :	10
3.3	UCHelper	12
3.4	UCComponent	13
3.4.1	Component Button :	13
3.4.2	Component Table :	13
3.4.3	Component Form :	15
3.4.4	UCForm Helpers :	15
3.5	UCBase, UC Ajax	16
4	UC Api Core	20
5	Quy tắc thiết kế giao diện trang Quản Trị.....	Error! Bookmark not defined.

Lịch sử sửa đổi

Phiên bản	Tác giả	Ngày thực hiện	Mô tả
1.0	Vũ Hữu Bảo	16/12/2024	Phiên bản 1

1 Tổng quan tài liệu

1.1 Mục đích của tài liệu

- Tài liệu này mô tả và hướng dẫn sử dụng các framework, thư viện, pattern, ... để xây dựng và phát triển Web cho người mới bắt đầu tham gia các dự án cùng UCVN.

1.2 Đối tượng đọc tài liệu

- IT student
- Web Front-end developer
- Web Back-end developer
- Full-stack web developer

1.3 Thông tin phát triển

1.3.1 Ngôn ngữ lập trình, thư viện, framework

- Ngôn ngữ lập trình : sử dụng 2 ngôn ngữ lập trình chính là C#, Javascript
- Framework sử dụng để phát triển Web:
 - ASP.NET : là một nền tảng ứng dụng web được phát triển và cung cấp bởi Microsoft, cho phép những người lập trình tạo ra những trang web động, những ứng dụng web và những dịch vụ web.
 - ASP.NET MVC : là một phần của ASP.NET dùng để thiết kế giao diện WEB, hỗ trợ mô hình **Model-View-Controller (MVC)**
 - 1. Model (Mô hình):**
 - Chịu trách nhiệm quản lý dữ liệu và logic nghiệp vụ.
 - Model giao tiếp với cơ sở dữ liệu hoặc các lớp xử lý dữ liệu khác.
 - 2. View (Giao diện):**
 - Chịu trách nhiệm hiển thị thông tin cho người dùng.
 - View chỉ xử lý việc hiển thị dữ liệu từ Model mà không chứa logic phức tạp.
 - 3. Controller (Điều khiển):**
 - Xử lý các yêu cầu của người dùng.
 - Nhận dữ liệu từ Model, gửi đến View để hiển thị.
 - Đóng vai trò trung gian giữa Model và View.

⇒ Là framework phát triển giao diện Web cho front-end developer
 - ASP.NET API : là một phần của ASP.NET dùng để xây dựng các dịch vụ dựa trên giao thức HTTP/HTTPS, hỗ trợ xây dựng các ứng dụng RESTful
 - ⇒ Là framework phát triển Web API cho back-end developer
- Thư viện :

- Phát triển front-end developer :
 - o Bootstrap5: thư viện thiết kế giao diện người dùng (sử dụng cho trang admin)
 - o Tailwind: thư viện thiết kế giao diện người dùng (sử dụng cho trang publish)
 - o JQuery: thư viện thao tác với HTML, CSS, và các sự kiện DOM
 - o DataTables: thư viện JS để tạo ra các bảng dữ liệu
- Phát triển back-end developer :
 - o Dapper: thư viện ORM (Object-Relational Mapping) cho .NET, dùng để thực hiện việc truy xuất dữ liệu tới CSDL

1.3.2 Cơ sở dữ liệu

- PostgreSQL: là một hệ quản trị cơ sở dữ liệu mã nguồn mở và miễn phí

1.3.3 UCVN helper

- UCVN helper: là một thư viện được phát triển bởi UCVN, bao gồm các hàm, API, component, ... dùng để hỗ trợ việc tái sử dụng lại mã nguồn, giảm thời gian phát triển và tuân thủ các quy tắc phát triển đã được quy định.

1.3.4 Công cụ và môi trường phát triển

- Công cụ quản lý mã nguồn :
 - Github (<https://github.com/>) : nền tảng web lưu trữ và quản lý các mã nguồn tập trung
 - Git Fork (<https://git-fork.com/>) : công cụ quản lý mã nguồn tại các máy trạm
- Hệ quản trị cơ sở dữ liệu postgresql :
 - Postgresql Server (<https://www.postgresql.org/>) : cài đặt PostgreSQL 15, port: 5432, username: postgres, password: Ucvn@2024
 - Dbeaver : giao diện người dùng đồ họa, để tương tác với các hệ quản trị cơ sở dữ liệu thực hiện các tác vụ quản lý và truy vấn dữ liệu
- Công cụ cho back-end developer :
 - .NET SDK8
 - Visual Studio 2022
- Công cụ cho front-end developer :
 - Visual Studio Code
 - Postman

2 Hướng dẫn lập trình API

2.1 Cấu trúc dự án

- Common : chứa các class, const, enum, ... dùng chung của dự án
- Controllers : chứa các class controller, dùng để tiếp nhận và xử lý các yêu cầu từ client thông qua các phương thức (GET, POST, PUT, DELETE)
- Dtos (Data transfer object) : chứa các class được sử dụng để truyền dữ liệu giữa các client – client hoặc client – server
- Entities : chứa các class và thuộc tính ánh xạ tới cơ sở dữ liệu
- Infrastructure : chứa các class thư viện tương tác với dự án
- Services : chứa logic nghiệp vụ của dự án

2.2 Quy tắc chung

2.2.1 Quy tắc đặt tên :

- Entity : snake_case
- Variable : snake_case
- DTO Entity : snake_case
- Class : CamelCase
- Method : CamelCase
- Route API : kebab-case

2.2.2 Quy tắc tạo Entity :

```
namespace Uc.Lib.Api.Entities
{
    [Schema("public")]
    public class uc_sample: CoreEntity<int>
    {
        public string name { get; set; }
    }
}
```

- Yêu cầu entity phải có schema nào
- Yêu cầu phải kế thừa từ lớp BaseEntity hoặc CoreEntity và xác định rõ kiểu dữ liệu của id thông qua generic

2.2.3 Quy tắc viết API :

- Mỗi class Entity sẽ tạo tương ứng Controller và Service để nhận yêu và xử lý logic (với các Entity ở dạng quan hệ không tạo Controller và Service)
- Yêu cầu các class Controller xử lý logic nghiệp vụ phải kế thừa từ lớp ApiControllerCore hoặc ApiControllerBase
- Yêu cầu tạo DTO kiểu dữ liệu trả về và tham số truyền vào khi viết API

2.3 UCHelper

- OSearch : class hỗ trợ tìm kiếm khi sử dụng phân trang ở phía server

- PagedRequest, PagedResponse : class hỗ trợ tính toán khi phân trang
- Danh sách các class hỗ trợ xử lý trong namespace UC.Core :
 - o DateTimeHelpers : hỗ trợ xử lý ngày tháng năm
 - o FileHelpers : hỗ trợ xử lý file
 - o GuidHelpers : hỗ trợ xử lý kiểu dữ liệu là Guid
 - o NumberHelpers : hỗ trợ xử lý kiểu dữ liệu là số
 - o ObjectHelpers : hỗ trợ xử lý đối tượng
 - Mapping: ánh xạ dữ liệu từ đối tượng này sang đối tượng khác với cùng thuộc tính và kiểu dữ liệu
 - o StringHelpers : hỗ trợ xử lý kiểu dữ liệu là chuỗi
 - o TreeHelpers : hỗ trợ xử lý dữ liệu dạng cây thư mục
 - ListToTrees: chuyển cấu trúc danh sách sang dạng cấu trúc cây thư mục
 - GetNodeFromTree : tìm 1 phần tử trong cây thư mục
 - InsertNodeIntoTree : thêm 1 phần tử vào cây thư mục
 - UpdateNodeIntoTree : sửa 1 phần tử trong cây thư mục
 - DeleteNodeInTree : xóa 1 phần tử trong cây thư mục
- Instance variable:
 - o IDateTimeProvider : cung cấp instance DateTime để lấy dữ liệu ngày hiện tại
 - o IUserProvider : cung cấp instance User đang tồn tại trong phiên làm việc

2.4 Dapper cơ bản

2.4.1 Truy vấn dữ liệu :

- Lấy danh sách dữ liệu :

```
public async Task<List<Entities.Ums.ums_role>> GetRoleByCode(string code)
{
    string sql = string.Empty;
    sql = "select * from ums.ums_role ur where ur.code = @code";
    return await _unitOfWork.Repository.QueryListAsync<Entities.Ums.ums_role>(sql, new { code });
}
```

- Lấy 1 dữ liệu :

```
public async Task<Entities.Ums.ums_user?> GetUserByUserName(string userName)
{
    string sql = string.Empty;
    sql = "select * from ums.ums_user uu where uu.user_name = @userName";
    return await _unitOfWork.Repository.QueryFirstAsync<Entities.Ums.ums_user?>(sql, new { userName });
}
```

2.4.2 Thao tác xử lý dữ liệu :

- Thêm dữ liệu

```
var receipt = new Entities.BTC.as_receipts();
receipt.id = StringHelpers.ShortId();
receipt.order_id = order[0].id;
receipt.code = await _receipt.GetCodeAsync(string.Empty, 7);
receipt.name = "Tạo từ đơn đặt " + order[0].name;
receipt.receipt_date = _dateTimeProvider.OffsetNow;
receipt.receiver = _userProvider.UserName;
receipt.created_by = _userProvider.UserName;
receipt.created_date = _dateTimeProvider.OffsetNow;
receipt.status = "1";

await _unitOfWork.Repository.ExecuteAsync("INSERT INTO public.as_receipts (id, order_id, code, name, receipt_date, receiver, created_by, created_date, status) " +
    "VALUES(@id, @order_id, @code, @name, @receipt_date, @receiver, @created_by, @created_date, @status);", receipt);
```

- Sửa dữ liệu :

```
private async Task<int> UpdateAmountReceipt(string receipt_item_id)
{
    int? count_issues = await _unitOfWork.Repository.QueryFirstAsync<int>("select sum(amount_receipt) from as_issues ai where ai.receipt_item_id = @receipt_item_id", new { receipt_item_id });
    if(count_issues == null)
    {
        count_issues = 0;
    }
    await _unitOfWork.Repository.ExecuteAsync("UPDATE public.as_receipt_items SET amount_receipt=@count_issues WHERE id=@receipt_item_id", new { count_issues, receipt_item_id });
    return count_issues.Value;
}
```

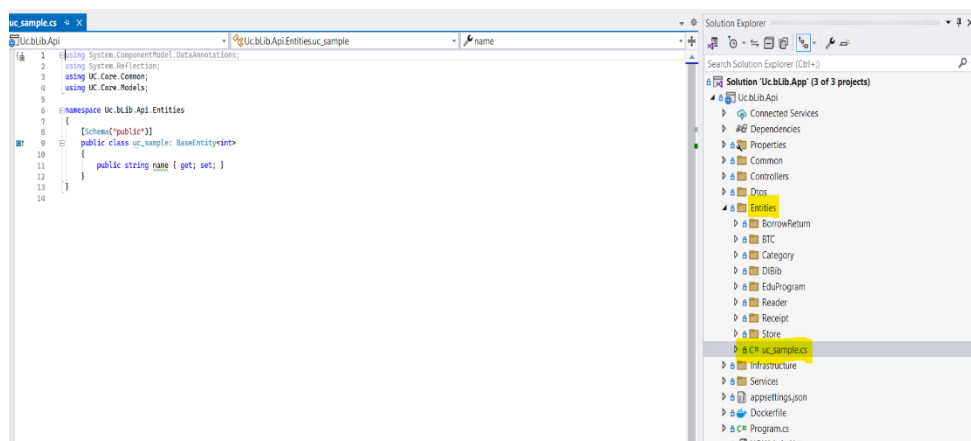
- Xóa dữ liệu :

```
sql = "delete from as_issues where id = @id;";
await _unitOfWork.Repository.ExecuteAsync(sql.ToString(), new { as_issues.id });
```

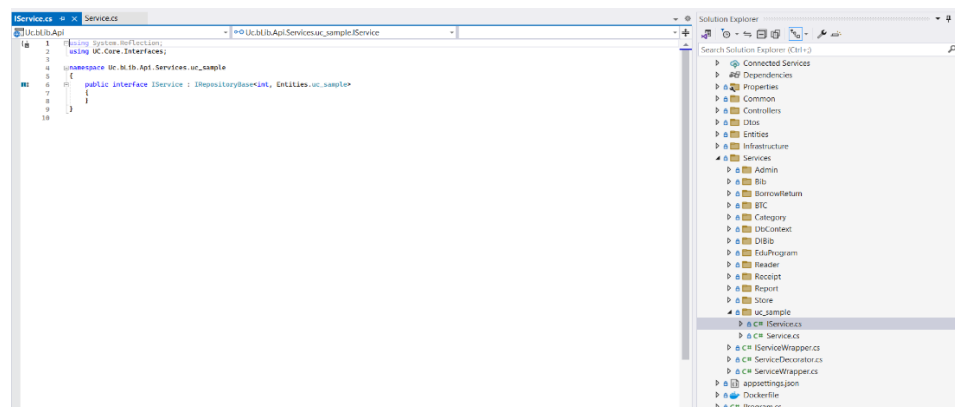
2.5 UC Query Builder

2.6 CRUD API cơ bản cho 1 Entity

- Bước 1: tạo entity ánh xạ tới table trong CSDL



- Bước 2: tạo service CRUD table trong CSDL

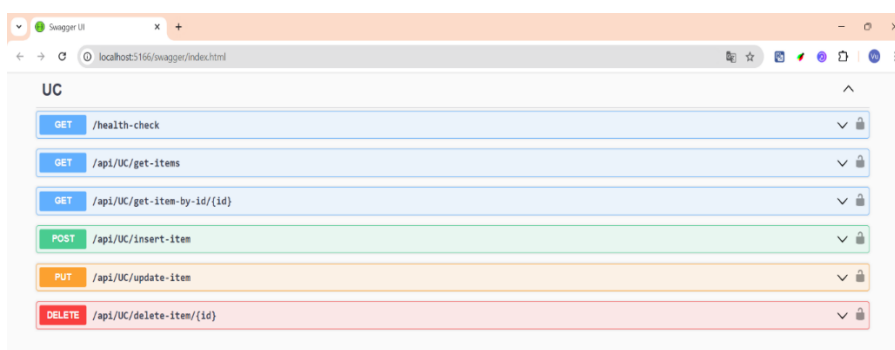


The screenshot displays the Visual Studio IDE with the `ServiceDecorator.cs` file open. The code defines a `ServiceDecorator` class that implements `IServiceDecorator` and implements the `Register` method. The `Register` method creates a `RepositoryBase` instance and adds it to the repository. The Solution Explorer on the right shows the project structure, including the `ServiceDecorator.cs` file.

```

1  using Microsoft.AspNetCore.Mvc;
2  using IUCCoreInterfaces;
3  using UC.KLib.Api.Services;
4  using Microsoft.Extensions.Caching.Memory;
5  using UC.KLib.Api.Context;
6  using Microsoft.AspNetCore.Authorization;
7
8  namespace UC.KLib.Api.Controllers
9  {
10     public class UCController : ApiControllerBase, IUCController
11     {
12         private readonly IServiceProvider _userService;
13         private readonly IServiceWrapper _service;
14         private readonly ILogger<UCController> _logger;
15         private readonly IMemoryCache _memoryCache;
16         public UCController(IServiceWrapper service, ILogger<UCController> logger, IServiceProvider userProvider, IMemoryCache memoryCache) : base(service, logger, userProvider, memoryCache)
17         {
18             _service = service;
19             _logger = logger;
20             _userProvider = userProvider;
21             _memoryCache = memoryCache;
22         }
23
24         [HttpGet("Health-Check")]
25         [AllowAnonymous]
26         public IActionResult HealthCheck()
27         {
28             return Ok("I'm fine !");
29         }
30     }
31 }

```



3 Hướng dẫn lập trình giao diện

3.1 Cấu trúc dự án

- Common : chứa các class, const, enum, ... dùng chung của dự án
- Controllers : chứa các class controller, dùng để tiếp nhận và xử lý các yêu cầu tới Views
- Dtos (Data transfer object) : chứa các class được sử dụng để truyền dữ liệu giữa các client – client hoặc client – server
- Pages/Shared : chứa bố cục của các Views
- Resources : chứa các file .rsx để làm đa ngôn ngữ
- ViewComponents : chứa các component dùng chung của dự án
- ViewModels : chứa các model của các view
- Views : chứa các file .cshtml để hiển thị giao diện người dùng
- Wwwroot/css : chứa file custom.style.css để chỉnh sửa css của dự án
- Wwwroot/js : chứa file custom.js để chỉnh sửa js của dự án và folder logic nghiệp vụ của dự án
- Wwwroot/json : chứa các file .json để cấu hình component

3.2 Quy tắc chung

3.2.1 Quy tắc đặt tên :

- Variable : snake_case
- Class : CamelCase
- Method : CamelCase
- Route : Kebab-case

3.2.2 Quy tắc code :

- Quy tắc tạo form cho giao diện :
 - Bước 1: Tạo Controller

```

using Microsoft.AspNetCore.Mvc;
using Uc.Lib.Web.Common;
using static Uc.Lib.Web.Common.Enums;

namespace Uc.Lib.Web.Controllers.Reader
{
    public class ReaderInfoController : Controller
    {
        private readonly IConfiguration _configuration;
        public ReaderInfoController(IConfiguration configuration)
        {
            _configuration = configuration;
        }

        [Route("ban-doc")]
        public IActionResult Index()
        {
            return View(ConstViewPaths.reader + "SearchReaderInfo/Index.cshtml");
        }

        [Route("tao-moi-ban-doc")]
        public IActionResult AddNew()
        {
        }
    }
}

```

- Đặt tên route cho View
- Xác định rõ ConstViewPaths thuộc phân hệ nào của chương trình

- Bước 2: Tạo View

```

@using Uc.Lib.Web.Common

@{
    ViewData["Title"] = "Chỉnh sửa bạn đọc";
    Layout = "~/Pages/Shared/_Layout.cshtml";
}

@section styles {
    <link rel="stylesheet" href="@ConstLocation.value/css/reader-detail.css" />
}

<div class="row">
    <div class="col-3 col-md-6">
        <ul class="breadcrumb">
        </ul>
    </div>
    <div class="col-9 col-md-6">
        <div class="text-end fixed-top-button">
            @await Component.InvokeAsync("UCAction", new
            {
                fileName = "Reader/EditReaderInfo/actions.json",
                code = "EditReaderInfo",
                type = "uc_action"
            })
        </div>
    </div>
</div>

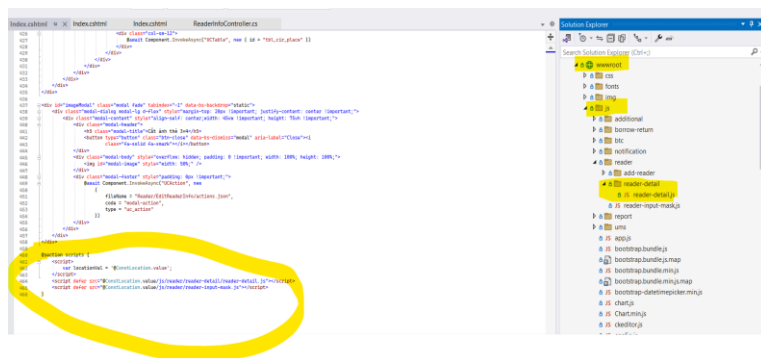
<div id="filter-inputs" class="filter-fields mt-4"></div>

<div class="row mt-2">
    <div class="col-sm-12">
        <div class="card" style="margin-bottom: 0px !important;">
            <ul class="nav nav-tabs nav-tabs-bottom" role="tablist">

```

- Xác định Layout của view
- Nếu View thuộc layout Admin mặc định phải có html breadcrumb mặc định

- Bước 3 : Tạo Javascript



- Tạo file JS giao diện import vào View

- Quy tắc nhận response khi gọi Ajax :

if (response.success) {

//xử lý logic

}

else {

if(response.message = "code_error_message") {

Toast().ShowToastError("Xử lý thất bại");

}

else if(response.message = "code_warning_message") {

Toast().ShowToastWarning("Cảnh báo");

}

else {

Toast().ShowToastError(response.message);

}

}

Chú thích: code_error_message, code_warning_message là mã thông báo là lỗi hoặc cảnh báo từ phía API trả về đã được xác định và thông báo sẽ được định nghĩa ở client

3.3 UCHelper

- Uc-Helper.js : cung cấp các hàm js dùng chung cho toàn dự án
- UcForm-Helper.js : cung cấp các hàm js xử lý hoặc vẽ Table, Controller, Select, ... cho toàn dự án
- Date-Util.js : cung cấp các hàm js xử lý ngày tháng

3.4 UComponent

3.4.1 Component Button :

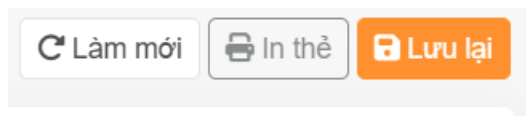
- Json config :

```
{
  "group_actions": [
    {
      "code": "AddNewReaderInfo",
      "actions": [
        {
          "id": "btn-refresh",
          "code": "",
          "type": "button",
          "title": "Làm mới",
          "isPrimary": false,
          "className": "",
          "classNameIcon": "fa-solid fa-rotate-right"
        },
        {
          "id": "btn-print-card",
          "code": "",
          "type": "button",
          "title": "In thẻ",
          "isPrimary": false,
          "className": "",
          "classNameIcon": "fa-solid fa-print"
        },
        {
          "id": "btn-add-reader",
          "code": "",
          "type": "button",
          "title": "Lưu lại",
          "isPrimary": true,
          "className": "",
          "classNameIcon": "fa-solid fa-floppy-disk"
        }
      ]
    }
  ]
},
```

- Html :

```
<div class="text-end fixed-top-button">
  @await Component.InvokeAsync("UCAction", new
  {
    fileName = "Reader/AddNewReaderInfo/actions.json",
    code = "AddNewReaderInfo",
    type = "uc_action"
  })
</div>
```

- o fileName : đường dẫn file cấu hình json
- o code : mã group_action
- o type : uc_action, uc_form, uc_form_search_list



Kết quả

3.4.2 Component Table :

- Json config :

```
"columns": [
  {
    "title": "Mã",
    "data": "code",
    "order": "desc",
    "colType": "text",
    "className": ""
  },
  {
    "title": "Tên",
    "data": "name",
    "colType": "text",
    "className": ""
  },
  {
    "data": null,
    "type": "dropdown",
    "defaultContent": [
      {
        "code": "Detail",
        "type": "modal",
        "title": "Sửa",
        "className": "action-edit",
        "classNameIcon": "fa-solid fa-pencil m-r-5 text-info"
      },
      {
        "code": "action-delete",
        "type": "button",
        "title": "Xóa",
        "className": "action-delete",
        "classNameIcon": "fa-regular fa-trash-can m-r-5 text-danger"
      }
    ]
  }
]
],
```

- Html :

```
<div class="row mt-2">
  <div class="col-md-12">
    <div class="card custom-card">
      <div class="card-header">
        Danh sách vai trò
      </div>
      <div class="card-body">
        <input type="hidden" id="hdKeyId" />
        @await Component.InvokeAsync("UCTable", new { id = "tbl_search" })
      </div>
    </div>
  </div>
</div>
```

- JS :

- Lấy dữ liệu cấu hình file json

```
AppState.jsonConfig = await Config().FormSearchList(
  "BTC/Order/SelectItemOrder.json"
);
```

- Gọi hàm vẽ Datatable

- Cách 1 : phân trang phía server

```
new UcFormHelpers().DataTableAjaxServerSide("#tbl_info",url_api,jsonConfig.columns,
searchField);
```

- Cách 2 : phân trang phía client

```
new UcFormHelpers().DataTableAjaxClientSide('#tbl_info', jsonConfig.columns, data);
```

Hiện thị: 10 bản ghi Tìm kiếm:

Mã	Tên
1a	2a
1	2

Hiện thị từ 1 đến 2 của 2 bản ghi < 1 >

Kết quả

3.4.3 Component Form :

- Json config :

```
{
  "form": {
    "code": "item_detail",
    "layout": {
      "rows": [
        {
          "cols": [
            {
              "code": "full_name",
              "label": "Họ và tên",
              "labelSize": 3,
              "controlType": "InputText",
              "controlSize": 9,
              "disabled": false,
              "required": true,
              "options": []
            }
          ]
        }
      ]
    }
  }
},
```

- Html :

```
<div id="add-container" class="add-container" style="display: none">
  @await Component.InvokeAsync("UCForm", new { fileName = "Ums/User/user_info_for_adding.json" })
</div>
<div id="edit-container" class="edit-container" style="display: none">
```

- o fileName : đường dẫn file cấu hình json
- o isFormSearch : true (folder: uc_form_search_list), false (folder: uc_form)
- o type : basic, advance, search

Thông tin bạn đọc

* Họ và tên	Ngày sinh dd/mm/yyyy
Số CCCD	Giới tính Nam
* Email	Điện thoại
Trình độ ---Chọn---	Nghề nghiệp ---Chọn---
Quốc tịch VN	Dân tộc Kinh
Nơi làm việc	Lớp
Địa chỉ	Ghi chú

Kết quả

3.4.4 UCForm Helpers :

- o UcFormHelpers().GetFormValues(form_code) => return form_fields
 - Lấy ra mảng tên trường và giá trị của form
- o UcFormHelpers().SetFormValues(form_code, values) => void

- Gán 1 đối tượng vào form
- `UcFormHelpers().FormFieldsToObject(form_fields) => return object`
 - Chuyển đổi mảng tên trường và giá trị thành đối tượng
- `UcFormHelpers().RefreshFormValues(form_code, ignore_ids = []) => void`
 - Làm mới giá trị trong form

3.5 UCBASE, UCAJAX

- **UCAjax** : Hỗ trợ các phương thức Get, Post, Put, Delete dựa trên Ajax
 - Method Get : `UcAjax().Get(url, hasToken)`
 - Method Delete : `UcAjax().Delete(url, hasToken)`
 - Method Post : `UcAjax().Post(url, data, hasToken)` (data là object hoặc formdata)
 - Method Put : `UcAjax().Put(url, data, hasToken)` (data là object hoặc formdata)

⇒ Response của các Method trả về có cấu trúc json sau :

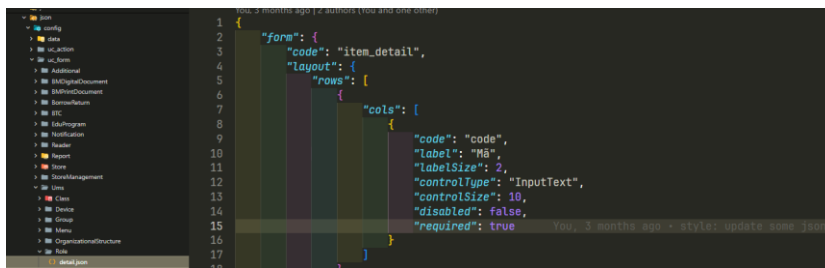
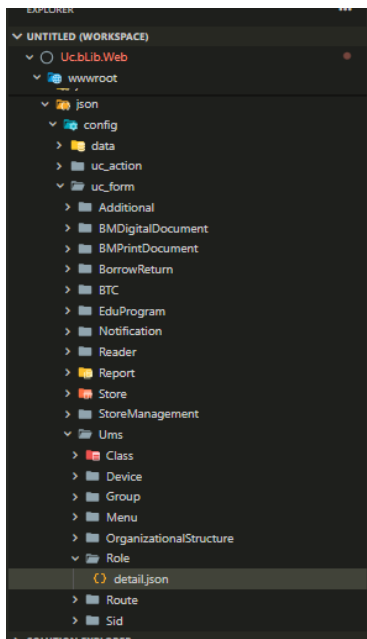
```
▶ data: {group_id: 1, group_name: "Tin tức nổi bật",...}
  message: "Xử lý thành công !"
  statusCode: 200
  success: true
```

- **UCBase** : Hỗ trợ tạo Form thêm, sửa, xóa, danh kết nối với API CRUD
 - Bước 1: Tạo controller cho service, module hiện tại.

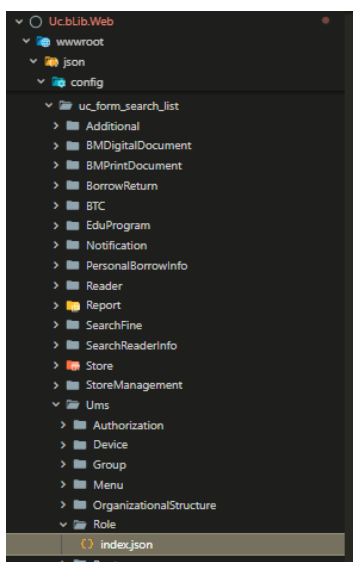
```
using Microsoft.AspNetCore.Mvc;
using Uc.bLib.Web.Common;

namespace Uc.bLib.Web.Controllers.Ums
{
    public class RoleController : Controller
    {
        public IActionResult Index()
        {
            return View(ConstViewPaths.ums + "Role/Index.cshtml");
        }
    }
}
```

- Bước 2: Tạo json form cho modal.



- Bước 3: Tạo json config cho table.

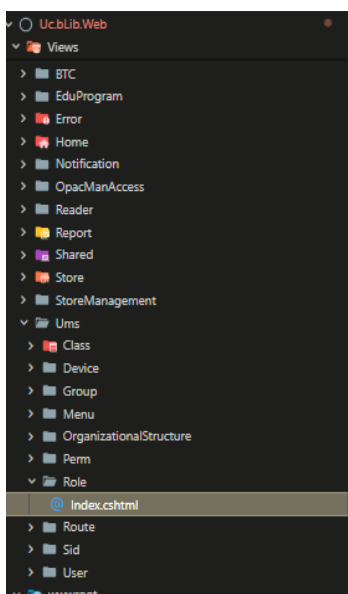


```

1  {
2    "columns": [
3      {
4        "title": "Mã",
5        "data": "code",
6        "order": "desc",
7        "colType": "text",
8        "className": ""
9      },
10     {
11       "title": "Tên",
12       "data": "name",
13       "colType": "text",
14       "className": ""
15     },
16     {
17       "data": null,
18       "type": "dropdown",
19       "defaultContent": [
20

```

○ Bước 4. Tạo view



```

1  @using UcLib.Web.Common
2
3  @{
4      ViewData["Title"] = "Vai trò";
5      Layout = "~/Pages/Shared/_Layout.cshtml";
6  }
7
8  <div class="row">
9      <div class="col-md-8">
10         <ul class="breadcrumb">
11             <li></li>
12         </ul>
13     </div>
14     <div class="col-md-4">
15         <div class="text-end">
16             @await Component.InvokeAsync("UCAction", new

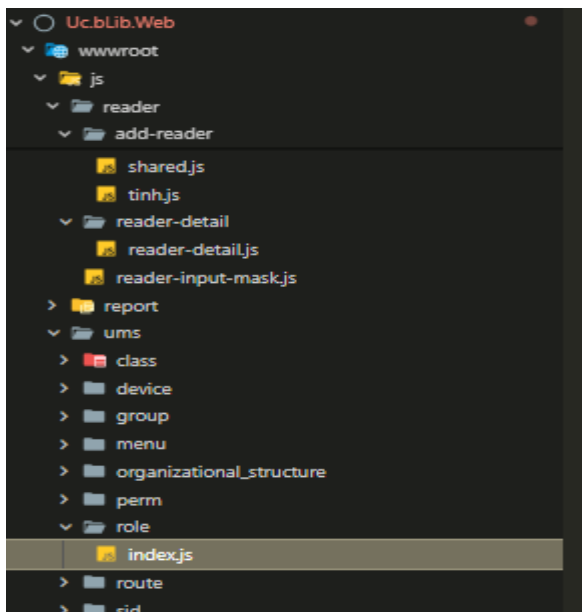
```

○ Bước 5: Tạo section scripts và tạo file js cho view

```

@section scripts {
    <script src="@ConstLocation.value/js/ums/role/index.js"></script>
}

```



- Bước 6: Triển khai UcBase cho script vừa tạo

```

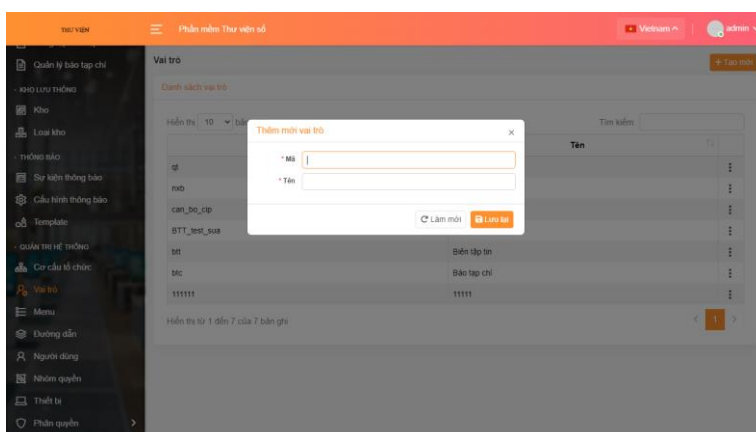
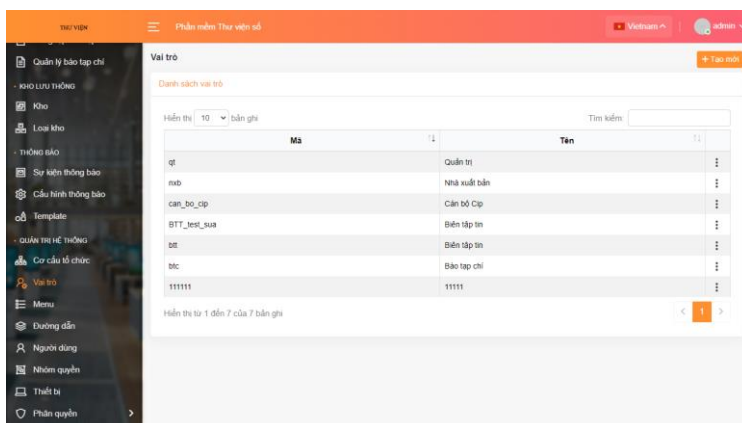
class RoleService extends UcBase {
  constructor(base_url, endPoint, jsonConfigPath) {
    super(base_url, endPoint, jsonConfigPath);
    super.configure({
      modalCode: "#modalDetail",
      hdKey: "#hdKeyId",
      formName: "Vai trò",
      formCode: "item_detail",
      labelCode: "#DetailLabel",
      needToBeUuid: true,
    });
  }

  init() {
    this.getList();
  }
}

$(document).ready(function () {
  const roleService = new RoleService(
    base_url_core,
    "Ums_Role",
    "Ums/Role/index.json"
  );
  roleService.init();
});

```

- Kết quả:



4 UC Api Core

- Link truy cập :
<https://ucvn.vn:5530/swagger/index.html?urls.primaryName=Core%20API%20-%20v1>
- Uc Api Core được tạo ra để sử dụng như các microservice. Mỗi microservice đảm nhiệm những tính năng nghiệp vụ dùng chung cho toàn bộ dự án. Các microservice UC đang hỗ trợ bao gồm :
 - Fms : các api quản lý File của hệ thống
 - Tải File lên
 - Tải File về
 - Xem File
 - Xóa File
 - Ums : các api truy cập, xác thực và quản lý người dùng của hệ thống
 - Quản lý truy cập của người dùng
 - Quản lý và phân quyền người dùng
 - Nms : các api quản lý thông báo tới người dùng (email, sms, push)
 - Quản lý mẫu thông báo được định nghĩa
 - Gửi thông báo

